

ElasticUMA: Reload-Correct OS-Managed Expert Caching for Apple Silicon

Naresh Prajapati

Independent Researcher

Abstract—Apple Silicon shares one physical memory pool across the CPU, GPU, filesystem cache, applications, compressor, and swap. A large routed-expert cache can therefore raise local hit rate while reducing end-to-end inference performance. ElasticUMA makes cold Metal-owned expert slots purgeable at synchronized prefill-layer and decode-token boundaries. Before any volatile slot is reused, the runtime relocks it; an empty return invalidates the logical mapping and triggers an exact positional reload before GPU binding. On an M1 Max with Qwen3.6-35B-A3B Q4, five-pair held-out tests show paired median gains of 7.3% over upstream-16 and 13.7% over upstream-96 under a 4 GiB co-tenant. Without synthetic pressure, gains are 16.9% and 17.6%. Fixed-96 physical footprint falls 31.6%. A second architecture, Gemma 4 26B-A4B, shows 45.3% and 26.0% paired median gains and a 35.3% footprint reduction. All 45 measured rows are admitted with exact within-protocol outputs. The results establish a two-architecture mechanism on one Mac; cross-chip, energy, and independent-reproduction claims remain open.

Index Terms—Apple Silicon, unified memory, mixture-of-experts inference, Metal, purgeable memory, expert caching, edge systems.

I. INTRODUCTION

Open-weight mixture-of-experts (MoE) models separate total capacity from active compute: a token routes through only a small fraction of the expert pool. FreeToken shows how that sparsity can turn a discrete-GPU personal machine into an elastic CPU-GPU serving platform [1]. On Apple Silicon, however, CPU and GPU do not have separate host and device pools connected by PCIe. They compete in one physical unified-memory pool with the display stack, page cache, applications, compression, and swap. The missing abstraction is therefore not simply offload; it is reload-correct physical elasticity inside one address space.

We begin with a falsification result. In a pinned Qwen3.6-35B-A3B Q4 runtime, increasing the explicit cache from 16 to 96 expert slots per layer raises hit rate from 48.98% to 90.13% but reduces paired median throughput by 10.54% and increases peak compressor growth by roughly 5 GiB. That misses a preregistered 15% threshold, so we reject a cache-size-only controller rather than lower the gate. The trace

instead reveals a stronger opportunity: preserve a large logical cache while making its cold physical pages reclaimable.

ElasticUMA realizes this idea with Metal purgeability. Each slot is allocated by Metal, cold slots transition to volatile only after GPU-safe boundaries, and every potential hit is relocked before use. Apple documents volatile resources as discardable caches that applications must lock and recreate or reload if reclaimed [2], [3]. ElasticUMA connects that API contract to an exact routed-expert cache.

The contribution is fourfold. First, we characterize why fixed expert capacity and physical memory are different objectives on Apple UMA. Second, we introduce a stable slot state machine that turns an empty Metal return into a normal exact cache miss. Third, we integrate the transition at true prefill-layer and decode-token safe points, preventing the full logical cache from materializing before it becomes reclaimable. Fourth, we provide complete held-out evidence against untouched small- and large-cache baselines on two MoE architectures, together with binary hashes, raw receipts, and negative mechanisms.

ElasticUMA: OS-managed expert residency with exact recovery

Cold pages become reclaimable; cache validity is re-established before every GPU reuse.

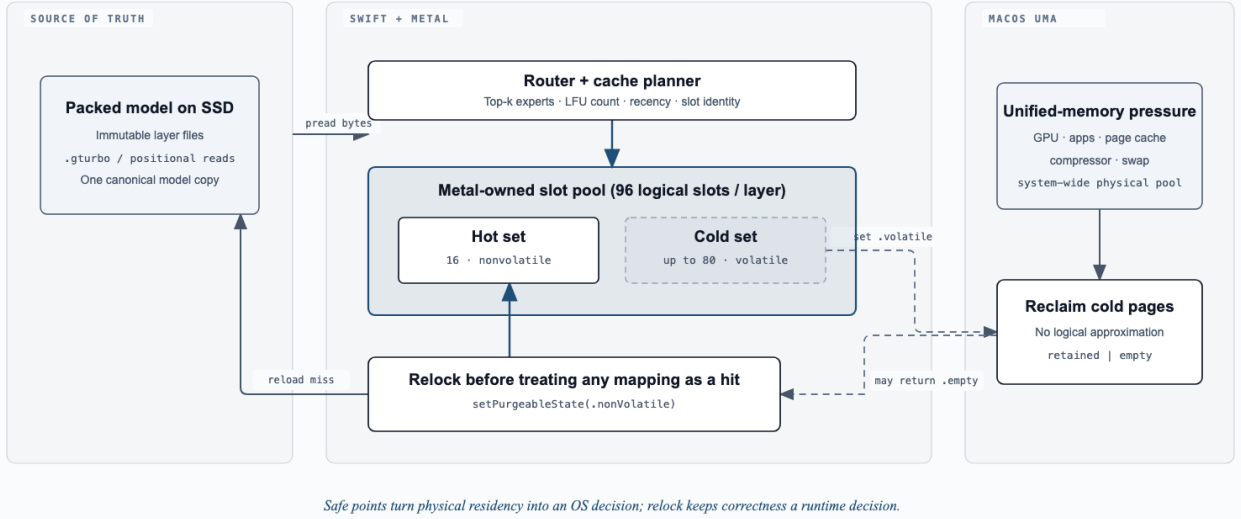


Fig. 1. ElasticUMA architecture. Metal-owned routed-expert slots preserve stable resources and CPU pointers. Hot slots stay nonvolatile; cold slots become volatile. A relock validates every potential hit, and an empty return triggers invalidation and exact positional reload from the immutable packed model.

II. PROBLEM AND DESIGN REQUIREMENTS

A. Why FreeToken's Resource Model Changes on Apple UMA

FreeToken keeps the complete expert pool in host memory, caches experts in VRAM, and uses calibrated host bandwidth and PCIe bandwidth to divide misses between transfer and CPU execution [1]. This is the correct abstraction for discrete GPUs. Apple UMA removes the PCIe copy boundary but not contention: explicit expert slots, file-backed source pages, KV or recurrent state, GPU allocations, and desktop processes all consume the same physical pool. A design that optimizes cache hit rate alone can force useful filesystem pages into compression and slow both I/O and GPU work.

B. Prefill and Decode Have Opposite Residency Shapes

Long-prompt prefill activates a broad union of experts. Decode activates eight experts per layer with temporal locality. FreeToken answers this split with full-layer double-buffering during prefill and semantic-aware caching during decode [1]. ElasticUMA follows the same challenge-to-design discipline but addresses a different bottleneck: a prefill callback after the entire prompt is too late, because all layer caches have already reached their physical peak. Cold pages must become reclaimable immediately after each layer's routed GPU work completes.

C. Reclamation Must Not Change Model Semantics

Purgeability separates virtual identity from physical validity. A slot may still be mapped to expert e even though macOS discarded its pages. The central invariant is: a volatile slot is never bound to a GPU encoder until a nonvolatile relock confirms that its prior contents survived. An empty return clears the expert identity and recency; the ordinary miss

path then reloads the complete expert blob. This changes latency and physical residency, never weights or routing.

III. ELASTICUMA DESIGN

A. Metal-Owned Per-Expert Slots

The upstream executor allocates anonymous memory and wraps it with bytesNoCopy Metal buffers [4]. Isolated probes show that marking these wrappers empty does not release the external allocation. ElasticUMA instead creates each `storageModeShared` buffer with `MTLDevice.makeBuffer`, then uses `buffer.contents()` as the pread destination. The Metal object and pointer remain stable across volatile, empty, nonvolatile, and refill states.

B. Selective Hot/Cold Residency

At a safe point, occupied slots are ranked by cumulative use count and recency. The configured hot subset (16 in the evaluated policy) is nonvolatile; remaining occupied slots transition to volatile. This creates 96 logical slots per layer but only a hot working set that the application insists remain physical. The OS may retain cold pages when headroom is plentiful or reclaim them when more important allocations arrive.

C. Reload-Correct State Machine

Slot state is nonvolatile, volatile, or empty. Planning first matches the requested expert identity. If the matching slot is volatile, the planner requests nonvolatile state under the cache lock. A retained prior state preserves the hit. An empty prior state increments recovery telemetry, clears identity and last-use state, and converts the request to a miss. Before pread overwrites any volatile/empty victim, that buffer is also made nonvolatile. Only a completed read publishes the new identity.

Correctness invariants. (I1) A GPU encoder binds a cached expert only after the slot is known nonvolatile. (I2) A logical expert identity is published only after a complete exact read of the immutable expert blob. (I3) Purgeability changes occur only after the last command using the resource has completed. (I4) Fixed residency never enters this state machine. Together, I1-I3 reduce an OS discard to an ordinary cache miss; I4 keeps the upstream baseline behaviorally isolated.

Lookup and recovery procedure. Under the cache lock, the planner first searches for the requested identity. A volatile match is relocked. If Metal reports retained contents, the access remains a hit. If Metal reports empty, the mapping is invalidated and the normal victim path is entered. The victim is made nonvolatile, `pread_exact` fills its stable pointer, and only then is the identity installed. At the next safe point, LFU count and recency select the hot nonvolatile subset; all other occupied slots become volatile.

D. GPU-Safe Transition Points

During chunked prefill, ElasticUMA marks the just-completed layer after its routed tiles and tail command buffer finish. During decode, the policy runs after the complete step and output head finish. These positions reuse synchronization already required for router readback and command completion. No live command buffer can reference a resource while its purgeable state changes. Fixed mode leaves the original path unchanged.

IV. IMPLEMENTATION

The implementation is runtime commit `ec84269d`, a 629-line change over pinned upstream commit `01f7d5e7`. It adds Metal ownership, slot-residency state and counters, model-level transitions, prefill/decode integration, public CLI and server controls, and focused unit/integration tests. The release interface is `--expert-cache-residency os-managed plus --expert-cache-hot-slots 16`.

A separate evidence plane records AC power, low-power/thermal state, system free memory, `vm_stat` compressor pages, swap, native `DispatchSource` memory-pressure events, process RSS, and `proc_pid_rusage` physical

footprint. RSS remains secondary because reclaimable pages may still appear resident. Runtime receipts record volatile transitions, relocks, retained relocks, empty recoveries, invalidated slots, and cumulative reload bytes.

The full upstream test command reports 593 tests: 588 pass; five pre-existing repacker CLI tests seek a debug executable named `TurboFieldfareRepack` while the package builds `slipstream-repack`. All new cache, inference, CLI, and server tests pass. Candidate and upstream release binary SHA-256 values are embedded in every archival result row.

V. EVALUATION

A. Questions and Protocol

We ask whether ElasticUMA (RQ1) beats the large fixed cache while reducing its physical pressure, (RQ2) remains competitive with the strongest small-cache fallback, (RQ3) works without a synthetic co-tenant, (RQ4) preserves exact output under real empty recoveries, and (RQ5) transfers to a second MoE architecture. Each positive protocol uses a previously unseen model-appropriate prompt, one excluded warmup, five measured repetitions in balanced order, greedy decoding, and one model worker.

Admission and frozen successor gate. A protocol passes only if all 15 planned rows are admitted, outputs match within the protocol, real empty recovery is observed, no row worsens swap or reaches critical pressure, and ElasticUMA provides a material physical-footprint or compressor benefit over fixed-96. Relative to either fallback, paired median slowdown and the bootstrap upper bound on slowdown must both be no worse than 10%. This 10% rule was frozen for these successor experiments; it is neither a universal product threshold nor the definition of a breakthrough.

Paired analysis. For repetition r and baseline b , gain is $g(r,b) = \text{throughput}(\text{candidate}, r) / \text{throughput}(b, r) - 1$. We report the median of five gains and a deterministic percentile interval from 10,000 bootstrap resamples (seed 104729). Medians summarize each arm's footprint, compressor growth, hit rate, and minimum free-memory reading. The intervals are descriptive engineering uncertainty for five fixed pairs; we do not pool prompts across models or interpret them as population-level significance tests.

TABLE I. Experimental platform and immutable artifacts

Item	Pinned value	Role
Host	Apple M1 Max, 24-core GPU, 32 GiB, macOS 26.5.2	Primary host
Qwen	Qwen3.6-35B-A3B Q4 @ 38740b84... [15]	256 experts, 40 layers
Gemma	Gemma 4 26B-A4B Q4 @ 0d77464e... [16]	128 experts, 30 layers
Upstream	slipstream @ 01f7d5e7...	Untouched fixed cache
ElasticUMA	runtime @ ec84269d...	OS-managed candidate
Runs	1 excluded warmup + 5 measured rows/arm	Balanced order, AC power

B. Negative Precursor Gate

Gate 1 sweeps Qwen cache sizes 16, 24, 32, 48, 64, and 96 with 30 admitted measurements. Hit rate rises monotonically, but throughput and headroom fall. The 16-to-96 paired

slowdown is 10.54% (bootstrap interval 8.94-10.84%), below a frozen 15% controller-authorization threshold. We retain this negative result and change the mechanism rather than the threshold.

Gate 1 fixed expert-cache sweep on M1 Max

Qwen3.6-35B-A3B Q4 · AC power · n=5 admitted runs per arm · identical outputs

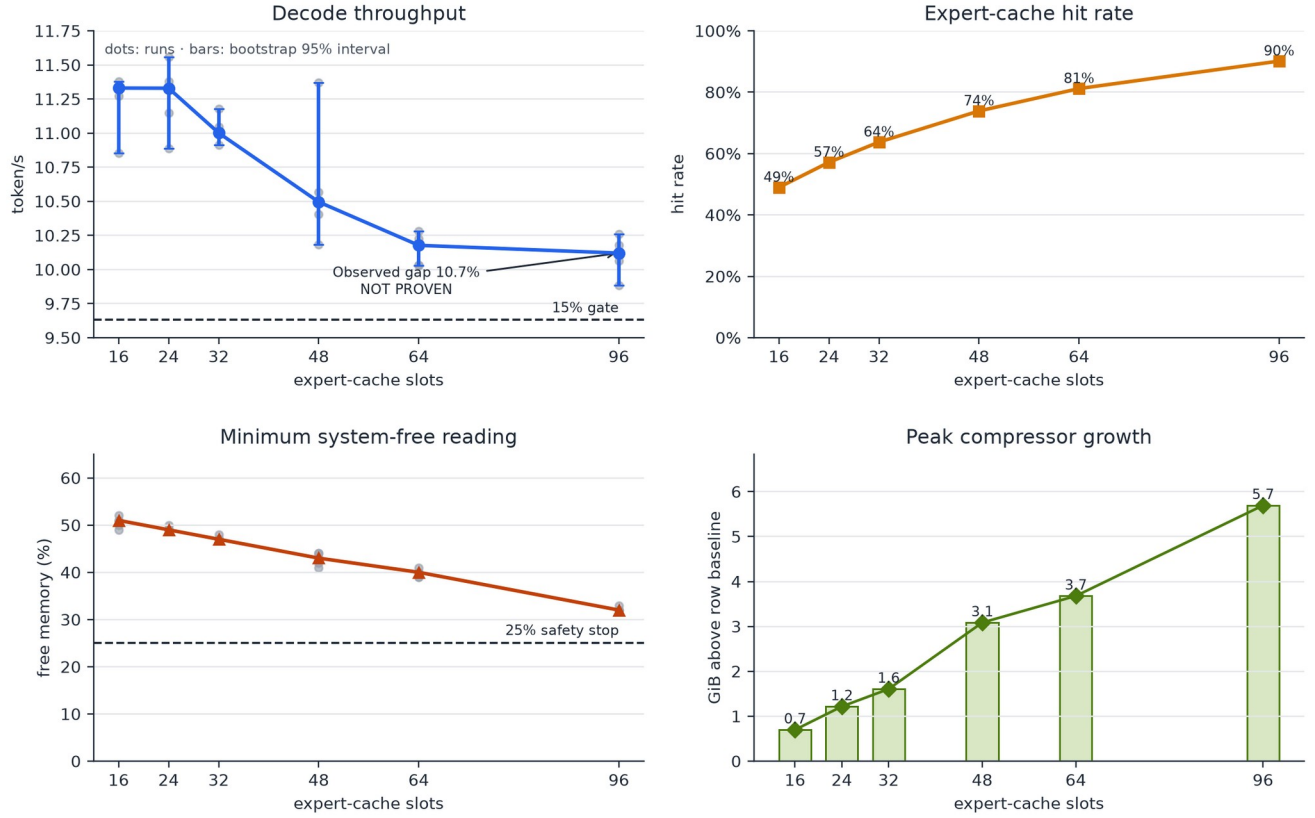


Fig. 2. Gate 1 fixed-cache characterization on Qwen3.6. Larger caches raise hit rate but reduce throughput, system-free memory, and compressor headroom. The repeatable gap misses the preregistered 15% controller gate.

C. Qwen Main Results

Under the 4 GiB co-tenant, ElasticUMA's paired median gain is 7.25% over upstream-16 and 13.67% over upstream-96. Without the co-tenant, gains are 16.91% and 17.59%. The candidate wins all ten no-pressure pairs and nine

of ten pressure-backed pairs. The only loss is 5.02%, while the candidate retains roughly 87% hit rate instead of the small cache's 48-50%. Relative to fixed-96, physical footprint falls 31.64% and 31.65% in the pressure and no-pressure protocols; compressor growth falls by 5.05 and 5.29 GiB, respectively. No arm grows swap.

Committed ElasticUMA versus untouched upstream fixed caches

M1 Max · Qwen3.6-35B-A3B Q4 · n=5 admitted runs per arm · identical outputs within each protocol

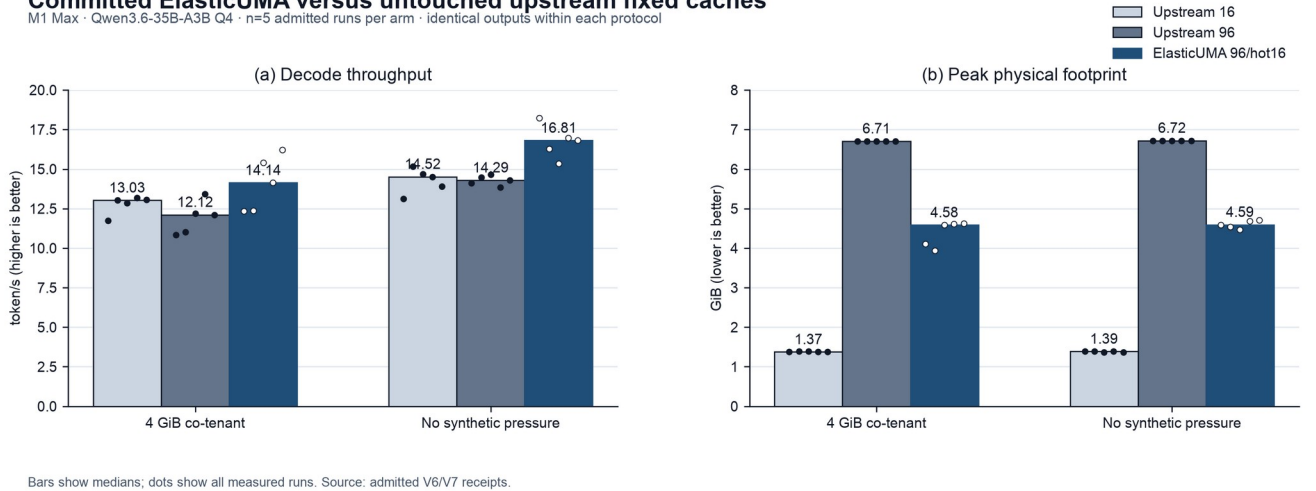


Fig. 3. Qwen3.6 main result. ElasticUMA preserves most 96-slot locality, beats both upstream fixed policies in median decode, and reduces fixed-96 peak physical footprint by approximately one third under both protocols. Bars are medians; dots are runs.

TABLE II. Qwen3.6 with 4 GiB controlled co-tenant

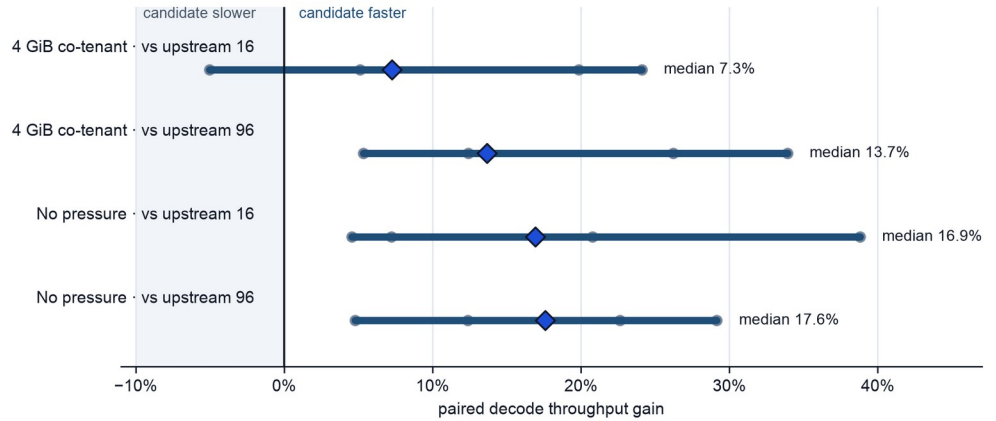
Policy	Decode tok/s	Hit rate	Peak footprint	Peak compressor	Min free
Upstream fixed 16	13.025	47.91%	1.374 GiB	0.675 GiB	57%
Upstream fixed 96	12.116	90.81%	6.705 GiB	5.890 GiB	38%
ElasticUMA 96/hot16	14.136	86.83%	4.584 GiB	0.840 GiB	44%

TABLE III. Qwen3.6 without synthetic pressure

Policy	Decode tok/s	Hit rate	Peak footprint	Peak compressor	Min free
Upstream fixed 16	14.520	49.82%	1.386 GiB	0.000 GiB	57%
Upstream fixed 96	14.293	90.58%	6.721 GiB	5.991 GiB	38%
ElasticUMA 96/hot16	16.807	87.50%	4.594 GiB	0.700 GiB	44%

Paired decode throughput gain by protocol and baseline

Candidate gain = (ElasticUMA - upstream) / upstream; five same-repetition pairs per row



Dots: measured pairs · diamond: median · navy interval: deterministic bootstrap 95% interval.

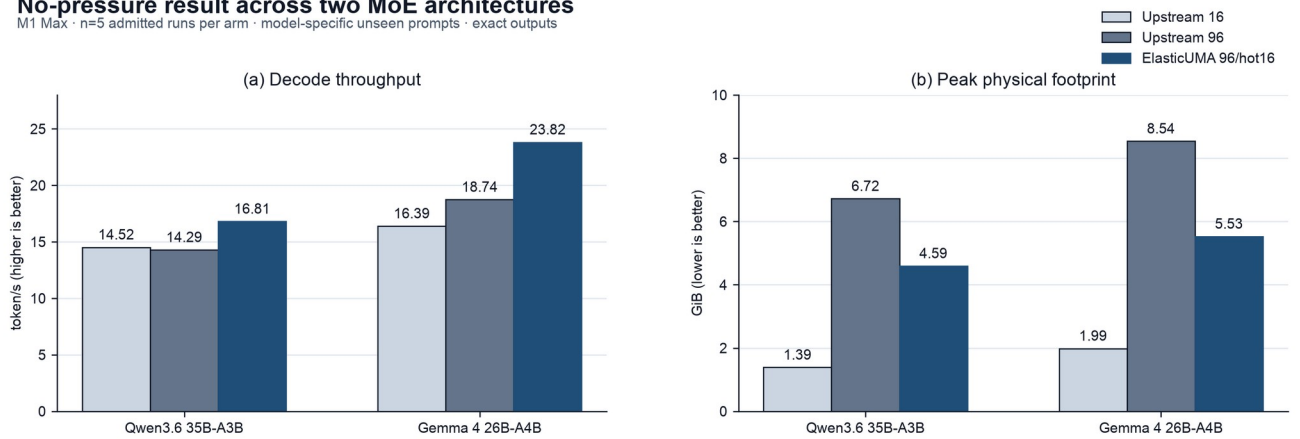
Fig. 4. Paired Qwen decode gains. Diamonds show paired medians and navy intervals show deterministic bootstrap 95% intervals. Every interval's lower end remains above -10%; three of four remain strictly above zero.

D. Cross-Architecture Result

Gemma 4 validates the mechanism against a different layer count, expert count, and routing distribution. ElasticUMA wins every pair: paired median gains are 45.30% over

upstream-16 (bootstrap interval 23.45-51.44%) and 26.01% over upstream-96 (11.88-28.55%). It retains 95.79% hit rate, reduces fixed-96 peak physical footprint from 8.538 to 5.527 GiB (35.26%), and reduces compressor growth from 5.114 to 1.073 GiB. No row grows swap.

No-pressure result across two MoE architectures
M1 Max · n=5 admitted runs per arm · model-specific unseen prompts · exact outputs



Bars show medians. Qwen uses 256 generated tokens; Gemma uses 128.

Fig. 5. No-pressure results across two MoE architectures. Absolute token/s values are not a model benchmark because prompts and generation lengths differ; within each model, ElasticUMA is the throughput leader and materially reduces fixed-96 footprint.

TABLE IV. Gemma 4 second-architecture validation

Policy	Decode tok/s	Hit rate	Peak footprint	Peak compressor	Min free
Upstream fixed 16	16.394	65.52%	1.986 GiB	0.000 GiB	69%
Upstream fixed 96	18.742	98.40%	8.538 GiB	5.114 GiB	50%
ElasticUMA 96/hot16	23.820	95.79%	5.527 GiB	1.073 GiB	55%

E. Exactness, Recovery, and Failed Alternatives

Every Qwen and Gemma candidate row observes nonzero empty recovery, yet each protocol has one response SHA-256 across all policies. Cumulative recovered bytes range from 7.9-10.8 GiB per Qwen run and 2.4-2.9 GiB per Gemma run; these counts include repeated reclamation and are not footprint. The equality result is text-level because the upstream CLI does not expose generated token IDs.

The path to the final design is itself an ablation. F_NOCACHE did not reduce slot footprint; direct mmap reduced RSS but was approximately 24x slower; raw and compressed MTLIO variants missed latency gates; lossless decode bandwidth was insufficient; a coarse one-arena-per-layer design collapsed Qwen effective hit rate to 1.3% and halved throughput; and a post-prefill-only slot policy failed a frozen footprint gate because the full cache had already materialized. Only selective per-slot state at the actual layer boundary passes.

F. Dense Control Diagnostic

The one-copy Qwen3.8-27B Q4 dense control runs under MLX 0.32.1 [17] and mlx-mlm 0.6.14, but it does not complete the frozen five-row safety protocol. One complete measured 128-token row reports 11.36 token/s, 17.96 GiB sampled peak footprint, 18.60 GiB MLX peak memory, 20% minimum free memory, and 151 MiB swap growth. A later row reaches 19% free and is terminated. We therefore treat it as capacity context, not a controlled speed baseline.

VI. RELATED WORK

A. Edge MoE Serving

FreeToken co-designs prefill, cache locality, CPU-GPU miss execution, agentic state reuse, and elastic VRAM management across heterogeneous CUDA machines [1], [18]. Its challenge-design-evaluation alignment motivates this paper's organization. The mechanism differs: FreeToken's source of truth is a CPU-resident expert pool and its q-star policy balances CPU bandwidth against PCIe transfer. ElasticUMA's source is an immutable packed file, and its novelty is OS-discardable Metal residency with exact validation inside a shared physical pool. Results are not compared across hardware.

MoE-Infinity establishes sparsity-aware expert caching on personal machines [5]; KTransformers provides CPU-GPU hybrid MoE inference [7]; MawForge and slipstream establish bounded materialization and native Mac expert streaming [4], [6]. These systems make larger-than-device models feasible, but fixed or runtime-owned cache capacity does not by itself make physical pages reclaimable to macOS.

B. Apple and On-Device Mechanisms

NPUMoE explores Apple Neural Engine execution [9], FusionML studies CPU-GPU co-execution boundaries on unified memory [10], and BaseRT demonstrates a native Metal runtime [11]. SliceMoE and ZipMoE reduce expert-transfer/storage cost through slicing and compression [8], [12]; MemSpec adapts speculation to memory [13]. These

mechanisms are complementary. ElasticUMA does not claim CPU-GPU co-execution, compression, slicing, or new kernels.

C. OS-Managed Memory

Apple explicitly describes volatile Metal resources as caches the OS may discard and requires applications to lock and recreate or reload them before access [2], [3]. ElasticUMA's contribution is the cache-validity protocol and phase integration that make this general graphics API safe and useful for routed model weights. Kernel-owned expert tiering is related [14], but our measured contract keeps model identity and recovery inside an explicit native runtime.

VII. LIMITATIONS AND THREATS TO VALIDITY

The positive evidence uses one M1 Max, two MoE architectures, batch size one, short held-out prompts, and one 4,096-token context limit. Cross-model prompts and generation lengths differ, so Figure 5 compares policies within each model rather than claiming one model is faster. The 4 GiB co-tenant is controlled, not a trace of a real desktop. No energy instrument is available, and native pressure events are coarse.

Five paired repetitions support deterministic bootstrap intervals but do not establish population-level generality. Another Apple generation, real application traces, longer contexts, batch/concurrency tests, and an independent operator are required before an all-Mac claim. Physical footprint is the primary memory metric; RSS includes reclaimable pages and can move in the opposite direction. The artifact excludes model weights, secrets, serial numbers, UUIDs, and personal paths from public bundles.

Output equality is deterministic response-text equality under greedy decoding. Because the pinned upstream CLI does not expose token IDs, the current adapter cannot assert token-level equality directly. Quality is not inferred from speed; all compared policies use the same checkpoint and weights, and the dense Qwen3.8 diagnostic is not used as a quality or performance baseline.

VIII. ARTIFACT AND REPRODUCIBILITY

The ElasticUMA harness and paper sources are in the isolated ElasticUMA repository; the runtime change is a separate clean repository with upstream history and an explicit read-only upstream remote. Reproduction configs pin model revisions, prompts, generation lengths, pressure limits, candidate/upstream binaries, and admission rules. Each run records a command hash, model/runtime identities, safe hardware fields, output hash, power/thermal context, memory time series, and native pressure receipt. Raw runs are immutable; derived analyses record their source SHA-256.

The admitted artifact SHA-256 prefixes are 5b23799f8653 (Qwen pressure), 06b78c1f2473 (Qwen no pressure), and 2b1c312e5660 (Gemma); the manifests carry the complete digests. The public release must include redacted receipts and scripts but never model files.

IX. CONCLUSION

A fixed expert cache confuses logical capacity with physical entitlement on Apple UMA. ElasticUMA separates them: the runtime retains a large logical cache, the OS decides whether cold pages remain physical, and a reload/reload protocol preserves exactness. Across Qwen3.6 and Gemma 4 on an M1 Max, this yields a repeatable Pareto improvement over both small and large upstream fixed caches while reducing the large cache's footprint by roughly one third. The evidence is strong enough for a scoped systems claim, not yet for all Apple hardware. The next decisive work is cross-generation reproduction, energy measurement, and evaluation on real concurrent desktop traces.

ACKNOWLEDGMENT

The author thanks the maintainers of slipstream, TurboFieldfare, MLX, llama.cpp, and FreeToken for releasing code and documentation that made direct comparison possible. An AI coding and writing assistant supported implementation, experimentation, figure generation, and editorial revision; the named author is responsible for validating all measurements, claims, citations, and submitted artifacts.

REFERENCES

- [1] S. Yang et al., 'FreeToken: Efficient Edge-Native MoE Serving with Bandwidth-Adaptive Execution,' arXiv:2608.16157, 2026.
- [2] Apple Inc., 'setPurgeableState(_);', Metal Developer Documentation, 2026. [https://developer.apple.com/documentation/metal/mtlresource/setpurgeablestate\(_\);](https://developer.apple.com/documentation/metal/mtlresource/setpurgeablestate(_);)
- [3] Apple Inc., 'Reducing the memory footprint of Metal apps,' 2026. <https://developer.apple.com/documentation/metal/reducing-the-memory-footprint-of-metal-apps>
- [4] D. Patel, 'slipstream,' source code, pinned at 01f7d5e7, 2026. <https://github.com/dwijenpatel/slipstream>
- [5] L. Xue, Y. Fu, Z. Lu, L. Mai, and M. Marina, 'MoE-Infinity: Efficient MoE Inference on Personal Machines with Sparsity-Aware Expert Cache,' arXiv:2401.14361, 2024.
- [6] C. Opie, 'MawForge: Memory-Bounded Expert Materialization for Local Mixture-of-Experts Inference,' arXiv:2607.09686, 2026.
- [7] H. Chen et al., 'KTransformers: Unleashing the Full Potential of CPU/GPU Hybrid Inference for MoE Models,' in Proc. 31st ACM SOSP, pp. 1014-1029, 2025, doi:10.1145/3731569.3764843.
- [8] Y. Choi et al., 'SliceMoE: Bit-Sliced Expert Caching under Miss-Rate Constraints for Efficient MoE Inference,' DAC, 2026, arXiv:2512.12990.
- [9] A. Benazir and F. X. Lin, 'Efficient Mixture-of-Experts LLM Inference with Apple Silicon NPUs,' arXiv:2604.18788, 2026.
- [10] O. Mohite, 'FusionML: Prefill, Not Decode - Mechanism and Boundaries of CPU+GPU Co-Execution on Unified-Memory Apple Silicon,' arXiv:2607.22785, 2026.
- [11] P. Rathnayaka, F. Waschkowski, and L. Wesemann, 'BaseRT: Best-in-Class LLM Inference on Apple Silicon via Native Metal,' arXiv:2607.00501, 2026.
- [12] Y. Yang, Y. Zhao, P. Yang, S. Wang, and Z.-H. Zhou, 'ZipMoE: Efficient On-Device MoE Serving via Lossless Compression and Cache-Affinity Scheduling,' ICML, 2026, arXiv:2601.21198.
- [13] E. Kim, Y. J. Jeon, and M. Han, 'MemSpec: Memory-Aware Runtime for Adaptive Draft Scheduling in Speculative Decoding on Edge Devices,' LCTES, pp. 180-192, 2026, doi:10.1145/3814943.3816174.
- [14] Y. Si, Y. Lin, D. Li, and J. Zhang, 'Who Should Own the Expert Cache? Kernel-Managed Tiering for Trillion-Parameter MoE Inference,' arXiv:2608.12103, 2026.
- [15] Qwen Team, 'Qwen3.6-35B-A3B model repository,' 2026. <https://huggingface.co/Qwen/Qwen3.6-35B-A3B>
- [16] Google, 'Gemma 4 Model Card,' 2026. https://ai.google.dev/gemma/docs/core/model_card_4
- [17] Apple Machine Learning Research, 'MLX: An Array Framework for Apple Silicon,' 2026. <https://github.com/ml-explains/mlx>
- [18] FlashML-org, 'FreeToken source repository,' 2026. <https://github.com/FlashML-org/FreeToken>